

Flax NNX 簡介

透過 JAX 建立神經網路

JAX 數值計算 · 吳至堯

大綱

章節	主題
Part 1	NNX 核心觀念：Module、Param、Rngs，以及怎麼跨回純 JAX
Part 2	訓練流程：寫一個 <code>train_step</code> 、train / eval 模式
Part 3	MNIST 示範：從零跑一個 CNN，看到 99% 為止
Part 4	Bonus：用 NNX 做 LoRA 微調

Setup

```
In [1]: import jax
import jax.numpy as jnp
from flax import nnx
```

Part 1 — NNX：Flax 的神經網路 API

為何需要 Flax？

- JAX 提供底層工具 — `jit`、`grad`、`vmap`、NumPy 風格的張量運算
- JAX 缺少抽象層
 - 只用 JAX 寫神經網路的話要處理很多雜事
 - 參數樹、RNG key、BatchNorm running stats、optimizer state 都得自行串接管理
- Flax = 把雜事整合好的官方框架
 - 工業：Gemma 開源、Gemini / Imagen / Veo 內部應用
 - 生態：HuggingFace Transformers、Scenic、Big Vision 都建立在 Flax 之上

從 nn 到 NNX: API 演進

- v1 `flax.nn` — 早期實驗，已淘汰
- v2 `flax.linen` (2020) — 功能完整，但寫起來不像 Python
 - 建模型分 `init` / `apply` 兩步，不像 PyTorch 一行建好
 - Dropout / BatchNorm 內部狀態要自己接進接出
 - 沒辦法對子模組直接呼叫方法
- v3 `flax.nnx` (2024) — Module 就是普通 Python 物件

- 參數即 屬性，可以印、修改或共享
- 寫起來約等於 PyTorch

Linen 還在，但官方文件已預設使用 NNX。

NNX 的設計原則

- **Pythonic** — 普通 class，屬性 可以改
 - `self.fc1 = nnx.Linear(...)` 跟 PyTorch 一樣
- **Explicit** — 維度、隨機數都要指定
 - `rngs = nnx.Rngs(0)` 統一管理數值
- **Eager** — `model = MyModel(rngs=...)` 直接取得可運行的模型
 - 舊版 Linen 要分 init / apply 兩階段
- **直接呼叫** — `y = model(x)` 就跑 forward
 - 不需要 `model.apply({'params': ...}, x)`

寫法是 PyTorch, 實際運行是 JAX

核心物件: Module、Param、Rngs

寫法和 PyTorch 幾乎一樣，三個物件講完就能開始寫:

- **nnx.Module** — 神經網路單位
 - 繼承 `nnx.Module`、forward 寫在 `__call__`、建構時把 `rngs` 傳進去
 - 建好直接呼叫 `y = model(x)` — 沒有 init / apply 兩階段
- **nnx.Param** — 「這個 array 是要訓練的」標籤
 - 貼上之後: 自動算梯度、被 optimizer 更新、進 checkpoint
 - 對應 PyTorch 的 `nn.Parameter`; 取值寫 `self.W[...]`
- **nnx.Rngs** — 隨機數派發器
 - JAX 沒有全域隨機數狀態，隨機數要顯式取得
 - `rngs.params()` 給參數初始化、`rngs.dropout()` 給 dropout
 - 同 seed 結果可重現, 方便 debug

Demo: 最小的 Module

把 `nnx.Linear` 放進自訂 Module。

流程：定義 → 建構 → forward

```
In [2]: class MyModel(nnx.Module):
        def __init__(self, din, dout, *, rngs):
            self.linear = nnx.Linear(din, dout, rngs=rngs)

        def __call__(self, x):
            return self.linear(x)

model = MyModel(4, 2, rngs=nnx.Rngs(0))
```

```
y = model(jnp.ones((1, 4)))
print("output:", y)
```

output: `[[ 0.05565438 -1.3324687 ]]`

內建層

跟 PyTorch 類似，涵蓋常見的 Layer：

- 全連接 / 卷積 — `nnx.Linear`、`nnx.Conv`、`nnx.ConvTranspose`
- 正規化 / 正則化 — `nnx.BatchNorm`、`nnx.LayerNorm`、`nnx.Dropout`
- 序列 / 注意力 — `nnx.LSTMCell`、`nnx.GRUCell`、`nnx.MultiHeadAttention`、`nnx.Embed`
- 激活函數 — `nnx.relu`、`nnx.gelu`、`nnx.silu`、`nnx.softmax`、`nnx.tanh`

沒覆蓋到的，可以直接把 `jax.numpy` 運算寫進 `__call__`。

Demo: 用 NNx 組 MLP

`self.fc1`、`self.bn`、`self.drop` — 跟 PyTorch 一樣，sub-module 就是屬性。

```
In [3]: class MLP(nnx.Module):
        def __init__(self, din, dhid, dout, *, rngs):
            self.fc1 = nnx.Linear(din, dhid, rngs=rngs)
            self.bn = nnx.BatchNorm(dhid, rngs=rngs)
            self.drop = nnx.Dropout(0.2, rngs=rngs)
            self.fc2 = nnx.Linear(dhid, dout, rngs=rngs)

        def __call__(self, x):
            x = self.fc1(x)
            x = self.bn(x)
            x = nnx.gelu(x)
            x = self.drop(x)
            return self.fc2(x)

model = MLP(784, 256, 10, rngs=nnx.Rngs(0))
y = model(jnp.ones((1, 784)))
print("MLP output shape:", y.shape)
```

MLP output shape: (1, 10)

Module 狀態: `self.x[...] += 1`

- Module 內部狀態可以直接改
 - 如 `self.x[...] += 1`
- 自訂狀態類別: 繼承 `nnx.Variable`
 - 如 `Param`、`BatchStat`、`Count` ...
- 透過標籤可依需求操作，如:
 - 只對 `Param` 求梯度
 - 只存 `BatchStat` 進 checkpoint
 - 凍結某類參數

```
In [4]: class Count(nnx.Variable):
        """Forward counter."""

class CountingLinear(nnx.Module):
    def __init__(self, din, dout, *, rngs):
        self.w = nnx.Param(
            jax.random.normal(rngs.params(), (din, dout))
        )
        self.count = Count(jnp.array(0))

    def __call__(self, x):
        self.count[...] += 1      # in-place update
        return x @ self.w[...]

model = CountingLinear(4, 2, rngs=nnx.Rngs(0))
_ = model(jnp.ones((1, 4)))
_ = model(jnp.ones((1, 4)))
_ = model(jnp.ones((1, 4)))
print(f"forward calls: {int(model.count[...])}")
```

forward calls: 3

函式式 API: split / merge / update

- 為什麼需要函式式 API ?
 - JAX 使用「純函式 + PyTree」機制
 - Module 是可變物件，JAX 無法處理
- 工具組：
 - `split` → 把 Module 拆成 (架構, 數值)
 - `merge` → 用 (架構, 數值) 重建 Module
 - `update` → 把新數值寫回原 Module
- 什麼時候會用到？
 - 存 checkpoint (Orbax 只認 PyTree，要先 split)
 - 模型匯出 / 跨環境部署 (架構跟值要分開存)
 - 混搭原生 `jax.scan` / `jax.pmap` (純 JAX 轉換看不懂 Module)

```
In [5]: # Reset model so we have a fresh nnx.Param-only example (the CountingLinear above
        # also has Count variables, which would need their own split filter).
demo_model = MyModel(4, 2, rngs=nnx.Rngs(0))

# split structure from values
graphdef, params = nnx.split(demo_model, nnx.Param)
print("graphdef type:", type(graphdef).__name__)
print("params leaves:", [leaf.shape for leaf in jax.tree.leaves(params)])

# Wrap with raw @jax.jit
@jax.jit
def pure_forward(graphdef, params, x):
    m = nnx.merge(graphdef, params)
    return m(x)

y = pure_forward(graphdef, params, jnp.ones((1, 4)))
print("pure-JAX forward output:", y)
```

```
graphdef type: GraphDef
params leaves: [(2,), (4, 2)]
pure-JAX forward output: [[ 0.05565438 -1.3324687 ]]
```

Part 2 — 訓練流程

NNX Transforms

- `@nnx.jit` — 用法跟 `@jax.jit` 一樣
 - 純 JAX 無法處理 Module，不能直接 `@jax.jit`，要自己 `split / merge`
 - 額外效果: 函式內對 `self.x` 的修改會傳回外面
- `nnx.value_and_grad` — 一次拿到 loss + 梯度
 - 寫法: `loss, grads = nnx.value_and_grad(loss_fn)(model)`
 - `grads` 結構自動對應 model 裡所有 `nnx.Param`
- `nnx.Optimizer(model, tx, wrt=nnx.Param)`
 - `wrt=nnx.Param` 在 Flax 0.11+ 必填, 指定要更新哪一類 Variable
 - 想做 LoRA? 把 `wrt` 換成 `LoraParam` 即可, 其他自動凍結

訓練迴圈

`train_step` 函式處理:

1. `loss_fn(m)` — 跑 forward、回傳 scalar loss
2. `loss, grads = nnx.value_and_grad(loss_fn)(model)`
3. `optimizer.update(model, grads)` — 更新權重
4. `return loss` — 給外層追蹤訓練曲線用

相對於 PyTorch 的簡化: 不需要 `zero_grad` 和分開的 `backward / step`。外面加 `@nnx.jit` 就能使用 GPU。

Demo: 完整的 `train_step`

```
In [6]: import optax

# Toy 4-input -> 2-class classifier
toy_model = MyModel(4, 2, rngs=nnx.Rngs(0))
optimizer = nnx.Optimizer(
    toy_model, optax.adam(1e-3), wrt=nnx.Param # wrt arg required in Flax 0.11+
)

@nnx.jit
def train_step(model, optimizer, batch):
    def loss_fn(m):
        logits = m(batch["x"])
        return optax.softmax_cross_entropy_with_integer_labels(
            logits, batch["y"]
        ).mean()

    loss, grads = nnx.value_and_grad(loss_fn)(model)
```

```

optimizer.update(model, grads)
return loss

# Random toy batch
key = jax.random.key(42)
toy_batch = {
    "x": jax.random.normal(key, (32, 4)),
    "y": jax.random.randint(key, (32,), 0, 2),
}

for step in range(5):
    loss = train_step(toy_model, optimizer, toy_batch)
    print(f"step {step}: loss = {float(loss):.4f}")

```

```

step 0: loss = 1.0290
step 1: loss = 1.0277
step 2: loss = 1.0265
step 3: loss = 1.0252
step 4: loss = 1.0240

```

跑一次 `train_step` 會自動更新:

- `BatchNorm` 的 running mean / variance
- `Dropout` 內部的 RNG 狀態
- `Optimizer` 的 step 計數、Adam 動量、lr schedule...

做法: NNX Module 是可變物件, 直接改狀態就好。Optimizer 也持有 model 的引用, 不用每步手動接 state。

對照 PyTorch

NNX 相比於 PyTorch 的簡化: 不需要 `zero_grad` (不累積梯度)、不需要分開的 `backward + step`。

步驟	PyTorch	Flax NNX
前向	<code>logits = model(x)</code>	<code>logits = model(x)</code>
損失	<code>loss = criterion(logits, y)</code>	寫在 <code>loss_fn(model)</code> 內回傳
反向	<code>loss.backward()</code>	<code>nnx.value_and_grad(loss_fn)(model)</code>
清梯度	<code>optimizer.zero_grad()</code>	不需要 (不累積)
更新	<code>optimizer.step()</code>	<code>optimizer.update(grads)</code>
編譯	<code>torch.compile(model)</code>	<code>@nnx.jit</code>

Train vs Eval 模式

為什麼要切換模式? 這兩種層在訓練 / 推論行為不一樣:

- **Dropout** — 訓練時隨機丟棄部分神經元、推論時全部保留
- **BatchNorm** — 訓練用 batch 統計、推論用累積的

NNX 的切換方法:

- `model.train()` / `model.eval()` — 跟 PyTorch 一樣，一次切整個模型
- 或在 `__call__` 顯式傳 `deterministic=True/False`

警告: 如果沒切 `eval`，Dropout 還在隨機丟棄，預測結果會無法重現。

Next step

剛剛 Part 2 用 toy data 跑了一個 4 維 → 2 類的 `train_step`。Part 3 用同樣的模式跑真資料 — 模型換成 CNN、資料換成 MNIST、迴圈跑 3,000 步看到 99% 準確率。

Pattern 不變: 定義 Module → `nnx.Optimizer` → `@nnx.jit train_step` → train / eval 切換。

Part 3 — MNIST

Step 1: 資料載入與預處理

- 使用 MNIST 標準下載源，包含 60K 訓練圖 + 10K 測試圖
- 28×28 灰階手寫數字
- 預處理: 像素 0–255 → 0.0–1.0, reshape 成 NHWC = (N, 28, 28, 1)

```
In [7]: import gzip, struct, urllib.request
from pathlib import Path
import numpy as np

DATA_DIR = Path("data")
MIRROR = "https://storage.googleapis.com/cvdf-datasets/mnist/"
FILES = {
    "train_x": "train-images-idx3-ubyte.gz",
    "train_y": "train-labels-idx1-ubyte.gz",
    "test_x": "t10k-images-idx3-ubyte.gz",
    "test_y": "t10k-labels-idx1-ubyte.gz",
}

def download_mnist():
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    for fname in FILES.values():
        dst = DATA_DIR / fname
        if not dst.exists():
            print(f" downloading {fname} ...")
            urllib.request.urlretrieve(MIRROR + fname, dst)

def load_images(path):
    with gzip.open(path, "rb") as f:
        magic, n, rows, cols = struct.unpack(">IIII", f.read(16))
        return np.frombuffer(f.read(), dtype=np.uint8).reshape(n, rows, cols)
```

```
def load_labels(path):
    with gzip.open(path, "rb") as f:
        magic, n = struct.unpack(">II", f.read(8))
        return np.frombuffer(f.read(), dtype=np.uint8)

download_mnist()
Xtr = load_images(DATA_DIR / FILES["train_x"]).astype(np.float32) / 255.0
Ytr = load_labels(DATA_DIR / FILES["train_y"]).astype(np.int32)
Xte = load_images(DATA_DIR / FILES["test_x"]).astype(np.float32) / 255.0
Yte = load_labels(DATA_DIR / FILES["test_y"]).astype(np.int32)
Xtr, Xte = Xtr[..., None], Xte[..., None] # add channel dim → NHWC
print(f"Train: {Xtr.shape}, labels: {Ytr.shape}")
print(f"Test: {Xte.shape}, labels: {Yte.shape}")
```

Train: (60000, 28, 28, 1), labels: (60000,)
 Test: (10000, 28, 28, 1), labels: (10000,)

Step 2: CNN 架構

卷積:

- **Filter** — 一個小窗滑過影像 · 抓局部 pattern
- **Feature map** — 每個 filter 對輸入的響應圖
- **Pooling** — 降採樣 · 給模型平移容忍度

示範用小型 CNN (28×28 → 10 個分類):

- Conv 1→32 (3×3) → ReLU → AvgPool (28→14)
- Conv 32→64 (3×3) → BN → ReLU → AvgPool (14→7)
- Flatten → Linear → Dropout → ReLU → Linear → 10 類

```
In [8]: class CNN(nnx.Module):
    def __init__(self, *, rngs):
        self.conv1 = nnx.Conv(1, 32, kernel_size=(3, 3), rngs=rngs)
        self.conv2 = nnx.Conv(32, 64, kernel_size=(3, 3), rngs=rngs)
        self.bn = nnx.BatchNorm(64, rngs=rngs)
        self.drop = nnx.Dropout(0.25, rngs=rngs)
        self.fc1 = nnx.Linear(7 * 7 * 64, 256, rngs=rngs)
        self.fc2 = nnx.Linear(256, 10, rngs=rngs)

    def __call__(self, x):
        x = nnx.avg_pool(nnx.relu(self.conv1(x)), (2, 2), strides=(2, 2))
        x = nnx.avg_pool(nnx.relu(self.bn(self.conv2(x))), (2, 2), strides=(2, 2))
        x = x.reshape(x.shape[0], -1)
        x = self.drop(nnx.relu(self.fc1(x)))
        return self.fc2(x)

model = CNN(rngs=nnx.Rngs(0))
# Sanity check on a single batch
out = model(jnp.asarray(Xtr[:4]))
print("CNN output shape (batch=4):", out.shape)
```

CNN output shape (batch=4): (4, 10)

Step 3: 訓練設定與訓練

- **Optimizer** — `optax.adamw(lr=5e-3)` + cosine LR 衰減
 - cosine decay: 學習率從 5e-3 沿餘弦曲線降到接近 0, 收尾更穩
- **Loss** — softmax cross-entropy (label 整數)
- **訓練**: 3,000 步, batch 32
- **評估**: 每 500 步看 train / test accuracy

```
In [9]: import time

LR, N_STEPS, BATCH_SIZE = 5e-3, 3000, 32
schedule = optax.cosine_decay_schedule(init_value=LR, decay_steps=N_STEPS, alpha

model = CNN(rngs=nnx.Rngs(0))
opt = nnx.Optimizer(model, optax.adamw(schedule, b1=0.9), wrt=nnx.Param)

def loss_fn(model, x, y):
    logits = model(x)
    return optax.softmax_cross_entropy_with_integer_labels(logits, y).mean()

@nnx.jit
def cnn_train_step(model, optimizer, x, y):
    loss, grads = nnx.value_and_grad(loss_fn)(model, x, y)
    optimizer.update(model, grads)
    return loss

@nnx.jit
def cnn_eval_step(model, x, y):
    preds = jnp.argmax(model(x), axis=-1)
    return (preds == y).mean()

rng = np.random.default_rng(0)
t0 = time.time()
for step in range(1, N_STEPS + 1):
    idx = rng.integers(0, len(Xtr), size=BATCH_SIZE)
    x, y = jnp.asarray(Xtr[idx]), jnp.asarray(Ytr[idx])
    loss = cnn_train_step(model, opt, x, y)
    if step == 1 or step % 500 == 0:
        model.eval()
        acc = float(cnn_eval_step(model, jnp.asarray(Xte[:2000]), jnp.asarray(Yt
        model.train()
        print(f" step {step:4d} loss={float(loss):.4f} test_acc={acc:.4f} ({

# Final full-test-set evaluation
model.eval()
preds = []
for i in range(0, len(Xte), 512):
    preds.append(np.array(jnp.argmax(model(jnp.asarray(Xte[i:i+512])), axis=-1))
preds = np.concatenate(preds)
test_acc = float((preds == Yte).mean())
print(f"\nFinal test accuracy: {test_acc:.4f} (wall time {time.time()-t0:.1f}s

step    1  loss=2.5737  test_acc=0.1100  (0.9s)
step  500  loss=0.1303  test_acc=0.9655  (8.1s)
step 1000  loss=0.0356  test_acc=0.9750  (15.7s)
step 1500  loss=0.0069  test_acc=0.9790  (23.0s)
```

```
step 2000  loss=0.0284  test_acc=0.9860  (30.4s)
step 2500  loss=0.0071  test_acc=0.9865  (37.6s)
step 3000  loss=0.0146  test_acc=0.9885  (44.9s)
Final test accuracy:  0.9909  (wall time 47.2s)
```

Step 4: 推論與結果視覺化

- 推論模式做預測 — `model.eval()` 關掉 Dropout, `jnp.argmax` 取最大機率類別
- 5×5 grid 看模型實際表現 — 綠框 = 正確, 紅框 = 錯誤

```
In [10]: import matplotlib.pyplot as plt

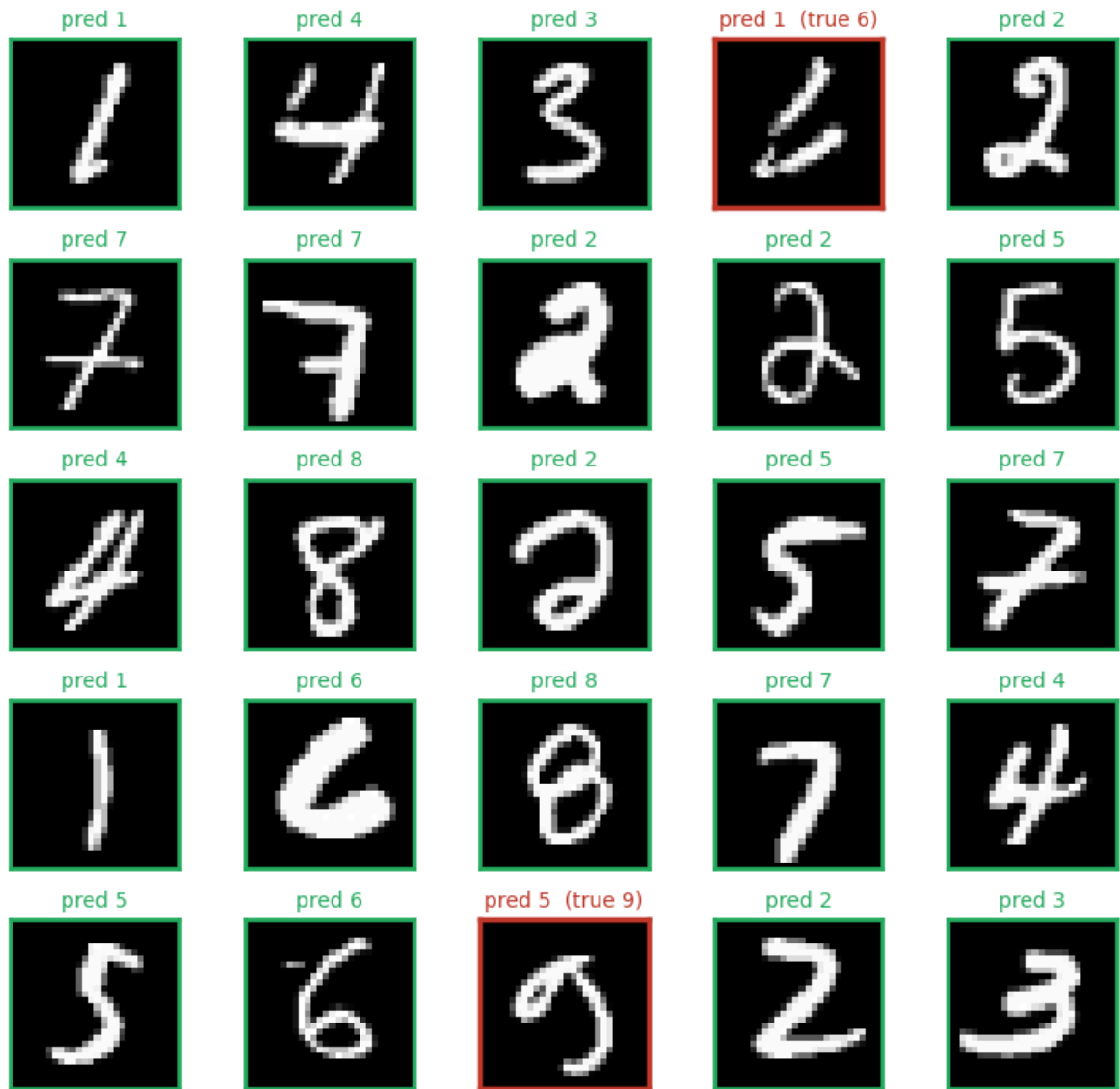
plt.rcParams["axes.unicode_minus"] = False

# Sample 23 correct + 2 wrong for the grid
rng_pick = np.random.default_rng(42)
correct_idx = np.where(preds == Yte)[0]
wrong_idx   = np.where(preds != Yte)[0]

n_c, n_w = 23, 2
picks_c = rng_pick.choice(correct_idx, size=n_c, replace=False)
picks_w = rng_pick.choice(wrong_idx,   size=min(n_w, len(wrong_idx)), replace=False)
pick = np.concatenate([picks_c, picks_w])
is_wrong = np.array([False]*len(picks_c) + [True]*len(picks_w))
perm = rng_pick.permutation(len(pick))
pick, is_wrong = pick[perm], is_wrong[perm]

fig, axes = plt.subplots(5, 5, figsize=(8, 8), facecolor="white")
for ax, idx, wrong in zip(axes.flat, pick, is_wrong):
    ax.imshow(Xte[idx].squeeze(), cmap="gray")
    true_label = int(Yte[idx])
    pred_label = int(preds[idx])
    if wrong:
        ax.set_title(f"pred {pred_label} (true {true_label})", fontsize=10, color="red")
        for sp in ax.spines.values():
            sp.set_edgecolor("#c0392b"); sp.set_linewidth(2.5)
    else:
        ax.set_title(f"pred {pred_label}", fontsize=10, color="green")
        for sp in ax.spines.values():
            sp.set_edgecolor("#27ae60"); sp.set_linewidth(2.0)
    ax.set_xticks([]); ax.set_yticks([])
fig.suptitle("MNIST predictions (green = correct, red = wrong)", fontsize=13, color="red")
plt.tight_layout(rect=(0, 0, 1, 0.98))
plt.show()
print(f"\nTest accuracy:  {test_acc:.4f}")
```

MNIST predictions (green = correct, red = wrong)



Test accuracy: 0.9909

核心概念對照: PyTorch ↔ Flax NNX

概念	PyTorch	Flax NNX
模組定義	<code>class M(nn.Module)</code>	<code>class M(nnx.Module)</code>
參數宣告	<code>nn.Parameter(tensor)</code>	<code>nnx.Param(jax_array)</code>
層	<code>nn.Linear(in, out)</code>	<code>nnx.Linear(in, out, rngs=rngs)</code>
Forward	<code>def forward(self, x)</code>	<code>def __call__(self, x)</code>
建模型	<code>model = M()</code>	<code>model = M(rngs=nnx.Rngs(0))</code>
拿參數	<code>model.parameters()</code>	<code>nnx.state(model, nnx.Param)</code>
反向	<code>loss.backward()</code>	<code>nnx.value_and_grad(loss_fn)(model)</code>
更新	<code>optimizer.step()</code>	<code>optimizer.update(grads)</code>
JIT	<code>torch.compile(model)</code>	<code>@nnx.jit</code>

概念	PyTorch	Flax NNX
存檔	<code>torch.save(state_dict)</code>	<code>nnx.split(model) → Orbax</code>

Part 4 — Bonus: 用 NNX 做 LoRA 微調

LoRA 是什麼

- 大模型微調的常見技巧
 - 原版微調訓練所有參數，慢且耗記憶體
 - LoRA 凍結原權重，只訓練兩個低秩小矩陣
- 數學: 原本 $y = Wx$ ，改成 $y = Wx + BAx$
 - A 、 B 是低秩矩陣，可訓練參數量降到 ~1%

為什麼 NNX 特別適合

- Module 是可變物件 → 直接替換屬性, 不用重建模型
 - `model.linear = LoraLinear(model.linear, rank=4)` 一行
- 給 A、B 新標籤: `class LoraParam(nnx.Variable)`
- `wrt=LoraParam` 只挑新標籤訓練 — 原權重自動凍結

同樣的操作在 PyTorch 通常需要寫不少 boilerplate

```
In [11]: class LoraParam(nnx.Variable):
          """Trainable, but separable from the base nnx.Param."""

          class LoraLinear(nnx.Module):
              def __init__(self, base: nnx.Linear, rank: int, *, rngs):
                  self.base = base # frozen
                  din, dout = base.in_features, base.out_features
                  self.A = LoraParam(jax.random.normal(rngs.params(), (din, rank)) * 0.01)
                  self.B = LoraParam(jnp.zeros((rank, dout)))

              def __call__(self, x):
                  return self.base(x) + (x @ self.A[...]) @ self.B[...]

          # Surgery – swap one attribute on the live model
          base_model = MyModel(4, 2, rngs=nnx.Rngs(0))
          base_model.linear = LoraLinear(base_model.linear, rank=2, rngs=nnx.Rngs(1))

          y = base_model(jnp.ones((1, 4)))
          print("LoRA-wrapped model output:", y)
          print(f"\nLoraParam A shape: {base_model.linear.A.shape}")
          print(f"LoraParam B shape: {base_model.linear.B.shape}")
          print(f"Trainable LoRA params: {base_model.linear.A[...].size + base_model.linear.B[...].size}")
          print(f"Frozen base params: {base_model.linear.base.kernel[...].size + base_model.linear.base.bias[...].size}")
```

LoRA-wrapped model output: `[[ 0.05565438 -1.3324687 ]]`

LoraParam A shape: (4, 2)

LoraParam B shape: (2, 2)

Trainable LoRA params: 12

Frozen base params: 10

資源 & References

- Flax 官方文件 — <https://flax.readthedocs.io/>
- NNX 基礎教學 — https://flax.readthedocs.io/en/stable/nnx_basics.html
- MNIST 教學 — https://flax.readthedocs.io/en/stable/mnist_tutorial.html
- 為什麼選擇 NNX ? — <https://flax.readthedocs.io/en/stable/why.html>
- GitHub — <https://github.com/google/flax>

Thank you